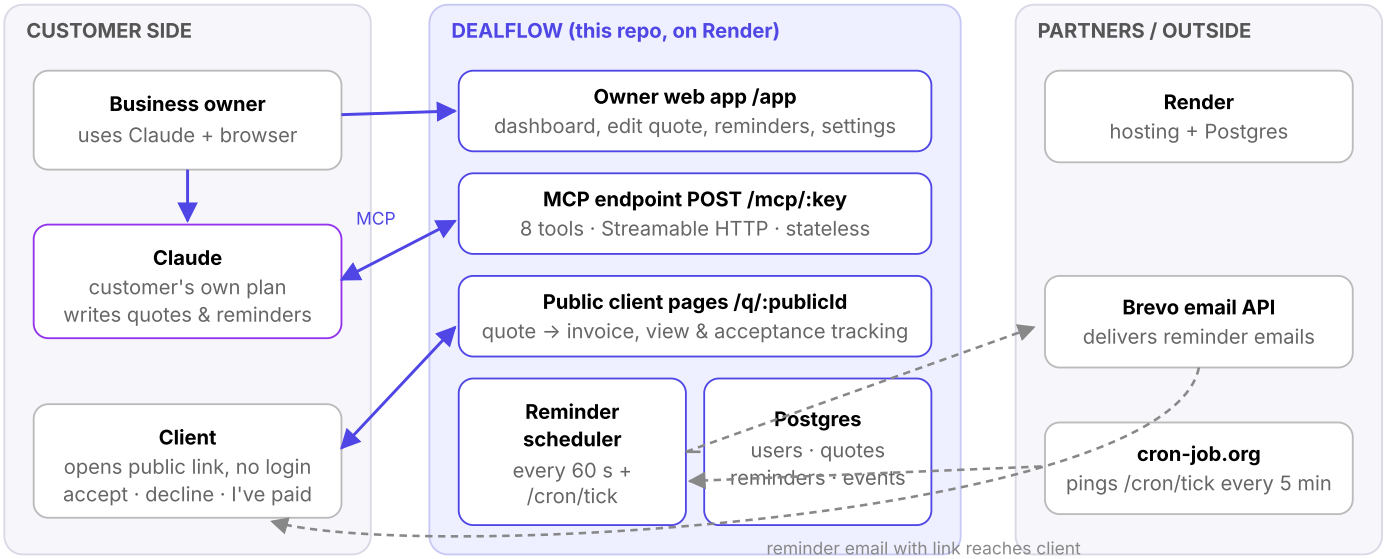
ARCHITECTURE & IMPLEMENTATION NOTES

Dealflow: how it's built

Claude does the thinking and writing. Dealflow does everything else. There are no AI keys in the code or configuration.



Data flow

1. **Owner → Claude:** "Quote Acme for a logo redesign...". Claude reads the tool descriptions Dealflow publishes over MCP and writes the content.
2. **Claude → Dealflow:** `POST /mcp/:key` → `create_quote`. The key in the URL identifies the account. Dealflow validates the input with zod schemas, computes totals and numbering (Q-0001), saves to Postgres, logs an event, and returns the public and edit links.
3. **Owner → Dealflow:** edits in `/app/quotes/:id`. Changes are saved and logged as field-level events. The client page always shows the latest version.
4. **Client → Dealflow:** `GET /q/:publicId` records the view, excluding the owner and link-preview bots. `POST …/accept` stores the name and timestamp, assigns INV-0001, sets the due date and shows payment instructions. `POST …/paid` flags "client says paid".
5. **Scheduler → client:** due reminders are claimed atomically (`FOR UPDATE SKIP LOCKED`), their placeholders (`{{amount}}`, `{{link}}` …) are filled in, and they're emailed through Brevo. Paid or declined quotes cancel pending reminders.
6. **Claude ← Dealflow:** `list_quotes` / `get_quote` return live status so Claude can report and draft follow-ups.

Our app vs. partner responsibilities

Who	Responsible for
Dealflow (built here)	Accounts and sessions; data model and storage; totals, tax, discount and numbering; status machine (draft → sent → viewed → accepted/invoice → paid, or declined); MCP tool definitions and validation; owner editing UI; public client pages; reminder scheduling, templating, sending and cancellation; activity log.
Claude (Anthropic, paid by the customer)	All language and reasoning: scope, line-item breakdown, pricing structure, terms, email and reminder copy, summarising status. Dealflow never calls an AI API.
Render	Runs the Node service and the managed Postgres; HTTPS; deploys from GitHub using `render.yaml`.
Brevo	Email delivery (free plan, verified single sender). Optional: without it, emails are recorded in-app.
cron-job.org	External ping every 5 minutes, so reminders fire and the free instance doesn't sleep.

Tools used

Runtime: Node.js 20+, Express 5, `@modelcontextprotocol/sdk` (Streamable HTTP server, stateless), `zod` (tool input schemas), `pg` (Postgres). **Frontend:** server-rendered HTML with about 120 lines of vanilla JS for the line-item editor, copy buttons and local time zones. No framework, no build step. **Security:** scrypt password hashing and HMAC-signed HttpOnly session cookies, both from Node's built-in crypto; everything is HTML-escaped; public IDs are random 12-character strings. **Infra:** Render (web service and Postgres), Brevo (email), cron-job.org. **Build tooling:** Claude Code helped write the code. That's allowed by the brief, and none of it runs inside the product.

Data model

Table	Key fields
users	email, password_hash, **mcp_key** (secret connector id), business name/address/tax id, payment instructions, default currency
quotes	public_id, seq (Q-n), invoice_seq (INV-n), client, items (JSONB), tax %, discount, scope, terms, payment terms, valid_until, due_date, status, view/accept/decline/paid timestamps
reminders	quote_id, send_at, subject, body (with placeholders), status (scheduled / sending / sent / failed / cancelled), delivery, error
events	quote_id, actor (Claude, owner, client, Dealflow), message, timestamp: the activity timeline

What was built vs. what is proposed

BUILT & LIVE	SIMULATED / MANUAL	PROPOSED PRODUCTION SYSTEM
MCP server, 8 tools, per-account connector URL	Payment collection: owner's UPI/bank text plus a client "I've paid" button; owner confirms	Razorpay/Stripe payment links, webhook auto-marks paid, 0.5% take rate
Owner dashboard and full editing, including reminders	Email: real only when a Brevo key is set; otherwise "recorded, email off"	Own sending domain (SPF/DKIM), bounce handling, WhatsApp Business reminders
Public quote and invoice pages, view tracking, accept/decline, print to PDF	Demo account's payment details are samples	GST-compliant invoice PDFs, e-sign audit trail, client portal
Reminder scheduler with auto-cancel; activity log	Connector auth is a secret URL	OAuth 2.1 connector auth, team seats and roles, audit logs
Live on Render with Postgres; auto-migrating schema	Free hosting tier with a keep-alive ping	Paid instances, read replicas, queue-based scheduler (e.g. pg-boss), monitoring

Assumptions

- Customers already have a Claude plan with custom connectors (MCP). Dealflow adds no AI cost.
- Small businesses in India mostly get paid by UPI or bank transfer, so showing payment instructions is enough for v1.
- One owner per account; amounts are in the quote's own currency with no FX conversion.

Limitations (v1)

- Render free Postgres expires after 30 days, and free instances sleep without the cron ping (a cold start of about 50 s could time out a Claude tool call).
- The connector URL is a bearer secret. It can be reset from the Connect page, except on the shared demo account, which is pinned for reviewers.
- No rate limiting, file attachments or multi-user teams yet. Dashboard totals are grouped per currency.

Next steps

1. Payment links with automatic reconciliation, which is the biggest value add and a revenue line.
2. WhatsApp reminders and per-client reminder preferences.
3. OAuth connector and listing in Claude's connector directory for one-click install.
4. Recurring retainers ("bill Acme ₹50k on the 1st every month") and simple receivables reports, which Claude can query.